ULACIT
UNIVERSIDAD LATINOAMERICANA
DE CIENCIA Y TECNOLOGÍA
COSTA RICA

**DESARROLLO DE APLICACIONES PARA LA COMPUTACIÓN EN LA NUBE**

Código 16-0550 | III Cuatrimestre 2026

# Avance 1: Propuesta y diseño

*Proyecto: Sistema web para consultorios dentales*

Estudiantes:
Daniel Cordero Calderón
David Ureña Aguilar
Axel Esteban Umaña Marín
Andrés Víquez Chavarría
Diego Rubí Salas

Grupo: 2

Profesora: Karol Leal

Fecha: 09 de octubre del 2026

Repositorio: https://github.com/durena-dev/clinica-dental-cloud

# Índice

Índice....2
Introducción y justificación....3
Problema empresarial y técnico....3
Relevancia en el contexto cloud-native....3
Valor agregado de la propuesta....4
Descripción funcional....5
Módulos principales....5
Roles del sistema....5
Casos de uso principales....6
Alcance del proyecto....7
Incluido en este cuatrimestre....7
Fuera del alcance....7
Supuestos y restricciones técnicas....7
Stack tecnológico y arquitectura....9
Principios para operar en la nube....10
Metodología de trabajo....11
Roles del equipo....11
Estrategia de branching en Git....11
Reuniones, comunicación y seguimiento....11
Modelo de datos (DER inicial)....13
Decisiones de diseño....14
Entidades....14
Relaciones y cardinalidad....15
Diagramas de flujo de procesos principales....16
Agendamiento de una cita....16
Atención y expediente clínico....17
Riesgos y medidas de control....18
Anexo A. Script SQL del modelo de datos....19

## Introducción y justificación

Se propone una aplicación web para apoyar la operación diaria de una clínica dental. La solución centraliza el registro de pacientes, la disponibilidad de los odontólogos, el catálogo de servicios, la agenda de citas y el expediente clínico, e incorpora notificaciones por correo electrónico y una bitácora de cambios que conserva la trazabilidad de las acciones importantes. Para este cuatrimestre se prioriza un producto funcional de alcance controlado: demostrar un flujo completo, desde el registro del paciente hasta la reserva de la cita y el registro de la atención.

### Problema empresarial y técnico

Las clínicas dentales pequeñas y medianas suelen administrar su información en herramientas separadas: la agenda en calendarios compartidos, los datos de pacientes en hojas de cálculo y las notas de atención en documentos o expedientes físicos. Esta fragmentación provoca duplicidad de datos, dificulta localizar antecedentes, aumenta el riesgo de agendar dos citas en el mismo horario y limita saber qué servicios presta cada odontólogo y qué espacios siguen libres.

El sistema propuesto resuelve el problema con una única fuente de información. Cada cita se relaciona con un paciente, un odontólogo y un servicio; antes de confirmarla, la aplicación consulta el horario habitual del profesional, sus bloqueos y las citas existentes. Cada cambio queda en una bitácora con su estado anterior y el nuevo, y al terminar la atención el odontólogo registra el diagnóstico y el plan de tratamiento en un expediente vinculado a la cita.

### Relevancia en el contexto cloud-native

La clínica necesita acceder a su agenda desde distintos dispositivos sin depender de un servidor instalado en una sola oficina. Por eso la aplicación se diseña como un servicio web sin estado, con configuración por variables de entorno, base de datos administrada, despliegues repetibles en contenedores y validaciones automáticas en el pipeline. Este enfoque permite

aumentar recursos cuando crece la demanda, recuperar el servicio con rapidez y publicar nuevas versiones de forma controlada, y aplica directamente los principios de The Twelve-Factor App vistos en el curso.

**Valor agregado de la propuesta**

El valor está en integrar agenda, disponibilidad profesional, expediente clínico, control de acceso y trazabilidad en una sola solución. La validación de horarios reduce las reservas duplicadas, los recordatorios apoyan la asistencia y la bitácora permite saber quién cambió una cita y cuándo. El diseño modular facilita incorporar después facturación, pagos o más sedes sin comprometer el alcance del cuatrimestre. Desde lo académico, el proyecto permite aplicar APIs REST, diseño relacional, seguridad, contenedores, servicios sin servidor y despliegue en la nube sobre un problema realista.

## Descripción funcional

La aplicación se organiza en módulos que corresponden con las áreas del modelo de datos. La primera versión se concentra en lo necesario para agendar y completar una consulta.

### Módulos principales

*Tabla 1. Módulos principales del sistema.*

| Módulo | Funciones principales | Entidades |
|---|---|---|
| Acceso y seguridad | Inicio de sesión, estado de la cuenta y autorización por rol. | users, roles |
| Pacientes | Registro, consulta y actualización de datos personales, contacto y consentimiento. | patients |
| Odontólogos | Perfil profesional, especialidad, servicios habilitados, horario semanal y bloqueos. | dentists, specialties, dentist_services, dentist_schedules, dentist_blocks |
| Servicios | Catálogo con duración, precio y estado. | services |
| Citas | Búsqueda de disponibilidad, creación, confirmación, reprogramación, cancelación y cierre. | appointments, appointment_logs |
| Expediente clínico | Consulta de antecedentes y registro de examen, diagnóstico, tratamiento y notas. | medical_records |
| Notificaciones | Confirmación de citas y recordatorios automáticos por correo. | notifications |
| Reportes | Panel administrativo con reportes de consultas por período, odontólogo, servicio y estado. | appointments, dentists, services |

### Roles del sistema

*Tabla 2. Roles del sistema.*

| Rol | Responsabilidad |
|---|---|
| Administrador | Gestiona usuarios, servicios, especialidades y parámetros generales, y consulta los reportes de consultas. |
| Recepcionista | Registra pacientes, consulta disponibilidad y administra citas dentro de sus permisos. |
| Odontólogo | Consulta su agenda, revisa datos del paciente y completa el expediente clínico. |
| Paciente | Consulta servicios y disponibilidad, solicita una cita y recibe confirmaciones. |

**Casos de uso principales**

**Registro de paciente.** La persona recepcionista registra los datos personales y de contacto. El sistema valida los campos obligatorios, evita identificaciones duplicadas y guarda el consentimiento. Si el paciente usará el portal, se vincula su registro con una cuenta de usuario.

**Agendamiento de cita.** El paciente o la recepción selecciona un servicio, un odontólogo y una fecha. El sistema cruza el horario habitual, los bloqueos y las citas existentes. Si encuentra un espacio válido, registra la cita, conserva el precio de referencia y programa la confirmación; de lo contrario, muestra horarios alternativos.

**Reprogramación o cancelación.** La recepción localiza la cita y solicita el cambio. El sistema verifica permisos y disponibilidad, actualiza el estado y registra en la bitácora el estado anterior, el nuevo, la fecha y el usuario responsable. La notificación pendiente se ajusta o se cancela según corresponda.

**Consulta de reportes.** La persona administradora elige un período y, si lo desea, un odontólogo o un servicio. El sistema consulta las citas, las agrupa por estado (atendidas, canceladas, no asistidas) y muestra los totales para apoyar las decisiones de la clínica.

**Atención y expediente clínico.** El odontólogo abre la cita desde su agenda, revisa los antecedentes y documenta el examen, el diagnóstico, el tratamiento y las notas clínicas. Al guardar, el sistema vincula el expediente con la cita y marca la atención como completada.

## Alcance del proyecto

### Incluido en este cuatrimestre

- Aplicación web responsiva para computadora y teléfono.
- Autenticación con JWT y autorización por roles.
- Mantenimiento de pacientes, odontólogos, especialidades y servicios.
- Configuración de horarios y bloqueos de los odontólogos.
- Agenda de citas con validación de conflictos y estados controlados.
- Expediente clínico básico asociado a una cita.
- Bitácora de cambios de estado de las citas.
- Confirmación y recordatorio de citas por correo electrónico (integración adicional).
- Panel administrativo con reportes de consultas por período, odontólogo, servicio y estado.
- Despliegue de una versión demostrable en la nube, con pipeline de CI/CD y documentación del repositorio.

### Fuera del alcance

- Facturación, pagos en línea, integración con aseguradoras y contabilidad.
- Odontograma gráfica avanzado y almacenamiento de radiografías.
- Recetas con firma digital o integración con farmacias.
- Aplicaciones móviles nativas para iOS o Android.
- Múltiples sedes con inventario y reglas independientes.
- Permisos configurables desde pantalla (los permisos se definen por rol en el código).
- Notificaciones por SMS o mensajería instantánea; analítica predictiva e inteligencia artificial.

### Supuestos y restricciones técnicas

- La primera versión atiende una sola clínica y usa datos ficticios en pruebas y demostración.
- Los usuarios tienen acceso a Internet y usan navegadores modernos.

- El despliegue depende de los límites de los planes gratuitos o educativos de los servicios cloud elegidos.
- Las credenciales, secretos y datos personales reales no se guardan en el repositorio ni en ambientes de prueba.

## Stack tecnológico y arquitectura

Se propone una arquitectura web modular con una API REST versionada (/api/v1) y documentación Swagger/OpenAPI. El backend será una sola aplicación organizada por módulos, para evitar la complejidad operativa de los microservicios, y los recordatorios de citas se ejecutarán como una función serverless programada. Así el equipo demuestra un servicio distribuido real sin disparar el alcance. La API expondrá endpoints REST para autenticación, pacientes, odontólogos, servicios, citas, expediente clínico y reportes, con validación de datos y códigos de estado HTTP adecuados.

*Tabla 3. Stack tecnológico propuesto.*

| **Capa** | **Tecnología** | **Motivo** |
|---|---|---|
| Frontend | React con Vite, React Router, Axios, React Hook Form y Tailwind CSS; diseño responsivo (mobile-first) | Stack recomendado por el curso; una SPA que consume la API y se adapta a móvil y escritorio. |
| Backend | ASP.NET Core 8 Web API (C#) con Entity Framework Core | API REST con validación, JWT y Swagger de forma nativa; opción permitida por el curso. |
| Base de datos | PostgreSQL gestionado en Supabase | Modelo relacional con integridad referencial y restricciones; alojada en la nube con plan gratuito. |
| Contenedores | Docker y Docker Compose | Entorno reproducible para desarrollo, pruebas y demostración; imagen del backend menor a 200 MB. |
| Servicios cloud | Azure for Students: Static Web Apps (frontend), Container Apps (API) y Functions (recordatorios); correo transaccional con SendGrid o equivalente | Despliegue en nube pública con escalado y pago por uso. |
| CI/CD | GitHub Actions | Lint, pruebas unitarias del backend, build de la imagen Docker y despliegue. |
| Gestión del proyecto | GitHub Projects | Backlog y tablero junto al código. |
| Calidad | ESLint, xUnit y Conventional Commits | Estilo consistente, pruebas automatizadas y mensajes de commit claros. |
| Comunicación | Microsoft Teams (reuniones) y WhatsApp (avisos urgentes) | Coordinación rápida y registro de acuerdos. |

**Principios para operar en la nube**

La configuración depende de variables de entorno y el repositorio no contiene contraseñas, tokens ni cadenas de conexión reales (Factor III). La base de datos y el servicio de correo se tratan como recursos externos que se conectan por configuración (Factor IV). Cada despliegue se construye desde una versión identificada del código mediante el pipeline (Factor V). La API no guarda estado en memoria: la autenticación usa JWT con expiración, de modo que cualquier instancia puede atender cualquier solicitud (Factor VI). Los registros se escriben en la salida estándar con formato estructurado (Factor XI) y Docker Compose mantiene los ambientes de desarrollo y producción lo más parecidos posible (Factor X).

En seguridad, las contraseñas se almacenan con hash, el acceso se controla por rol con el principio de mínimo privilegio, todas las entradas se validan en frontend y backend, CORS se configura solo para los orígenes autorizados y los ambientes usan credenciales distintas. Los datos clínicos no aparecen en los logs.

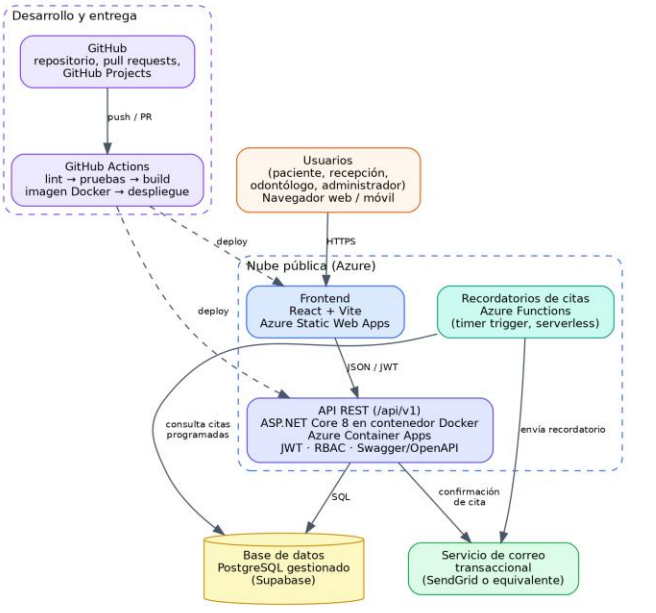



*Figura 1. Arquitectura cloud propuesta.*

# Metodología de trabajo

## Roles del equipo

Cada integrante tiene un rol principal, pero todos revisan cambios y conocen el flujo completo.

*Tabla 4. Roles del equipo de trabajo.*

| Integrante | Rol principal | Responsabilidades |
|---|---|---|
| David Ureña Aguilar | Coordinación y producto | Backlog, acuerdos, seguimiento y preparación de la exposición. |
| Daniel Cordero Calderón | Backend y seguridad | API, autenticación, autorización y reglas de negocio. |
| Axel Esteban Umaña Marín | Frontend y experiencia | Interfaz, formularios, validaciones y accesibilidad. |
| Andrés Víquez Chavarría | Datos y calidad | Migraciones, datos de prueba, pruebas automatizadas y apoyo al despliegue. |
| Diego Rubí Salas | Automatización de Despliegues | Implementar y mantener pipelines de automatización para integrar, probar y desplegar software. |

## Estrategia de branching en Git

Se usará un flujo basado en ramas cortas (GitHub Flow). La rama main está protegida: no admite push directo y exige pull request con al menos una aprobación de otro integrante y con el pipeline en verde. Cada tarea se desarrolla en una rama con nombre descriptivo, por ejemplo, feature/agenda-disponibilidad o fix/validacion-fecha. Los commits son pequeños y siguen Conventional Commits (feat:, fix:, docs:, chore:). Las entregas se marcan con tags: avance-2 y v1.0-final.

## Reuniones, comunicación y seguimiento

El equipo se reúne dos veces por semana en sesiones breves para asignar tareas y detectar bloqueos, y cierra cada semana con una revisión que muestra el incremento funcional, actualiza el tablero y confirma los compromisos siguientes. Las decisiones importantes quedan documentadas en GitHub Projects y en el repositorio. Teams se usa para las reuniones y WhatsApp para avisar

ausencias y resolver bloqueos urgentes. Las ausencias y conflictos se manejan según el contrato de equipo, que se entrega firmado junto con este documento.

## Modelo de datos (DER inicial)

El modelo relacional tiene 13 entidades pensadas para PostgreSQL y cubre seguridad, pacientes, odontólogos, servicios, horarios, bloqueos, citas, expedientes, notificaciones y bitácora. La Figura 2 muestra las entidades con sus llaves y la cardinalidad de cada relación (pata de gallo); la imagen completa y el archivo editable (DBML para dbdiagram.io) se entregan junto con el documento. El script SQL completo se incluye en el Anexo A y en el repositorio (database/schema.sql).

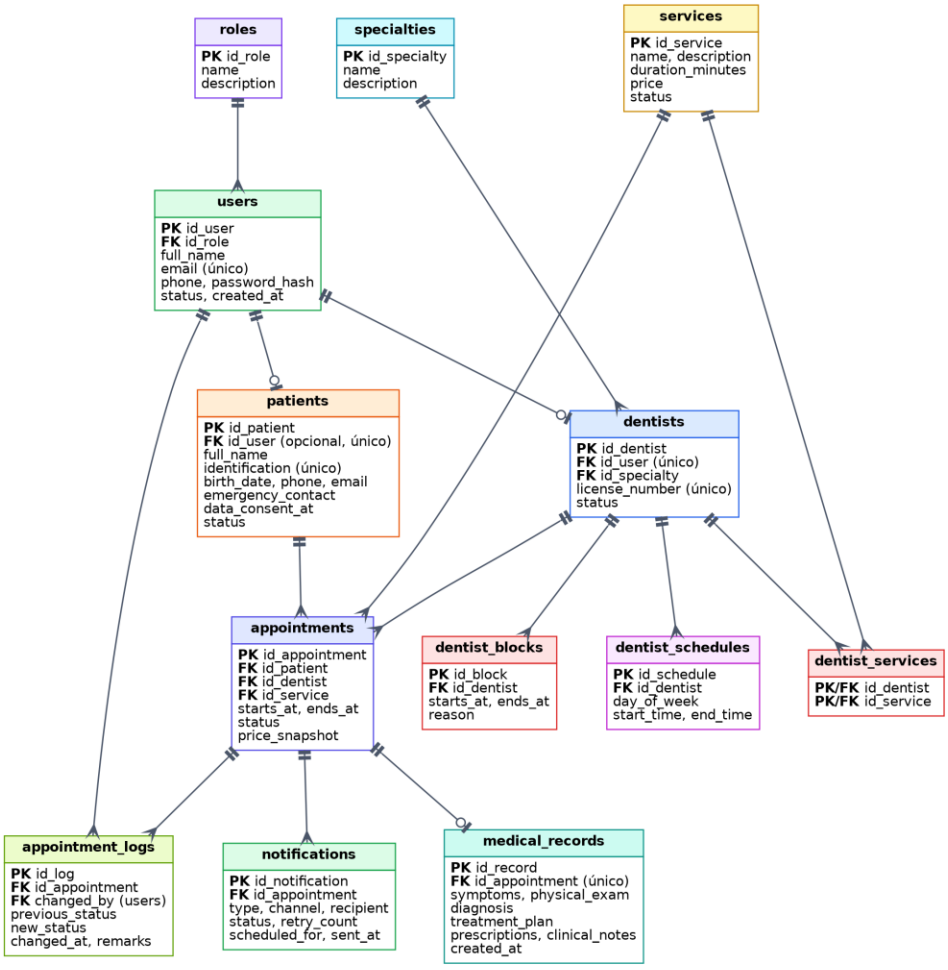



*Figura 2. Diagrama entidad-relación inicial.*

**Decisiones de diseño**

- **Un rol por usuario.** Cada usuario tiene un solo rol (users.id_role); los permisos se resuelven por rol en la API.
- **Tercera forma normal.** El expediente y las notificaciones se relacionan solo con la cita; el paciente y el odontólogo se obtienen a través de ella, sin duplicar datos.
- **Sin citas solapadas.** Se valida dentro de una transacción y se refuerza con una restricción de exclusión en PostgreSQL sobre odontólogo y rango de tiempo (appointments_no_overlap).
- **Servicios habilitados.** La cita referencia el par (odontólogo, servicio) de dentist_services, de modo que no se puede agendar un servicio que el profesional no tiene habilitado.
- **Fechas, estados e integridad.** Inicio y fin de citas y bloqueos son timestamptz con la restricción ends_at > starts_at; los estados usan CHECK y las llaves foráneas usan ON DELETE RESTRICT para conservar el historial clínico.
- **Índices.** Índices únicos en users.email y patients.identification, e índice por (id_dentist, appointment_date) para consultar disponibilidad.
- **Logs fuera de la base de datos.** Los errores se escriben en la salida estándar y no en una tabla, para no guardar trazas con datos sensibles.

**Entidades**

*Tabla 5. Entidades del modelo de datos.*

| Entidad | Llave primaria | Llaves foráneas | Propósito |
|---|---|---|---|
| roles | id_role | — | Roles del sistema (administrador, recepcionista, odontólogo, paciente). |
| users | id_user | id_role | Cuentas de acceso: nombre, correo, teléfono, credenciales y estado. |
| patients | id_patient | id_user (opcional) | Datos personales, contacto y consentimiento del paciente. |
| specialties | id_specialty | — | Catálogo de especialidades odontológicas. |
| dentists | id_dentist | id_user, id_specialty | Perfil profesional; el nombre y el contacto viven en users. |
| services | id_service | — | Catálogo con duración, precio y estado. |

| Entidad | Llave primaria | Llaves foráneas | Propósito |
|---|---|---|---|
| dentist_services | id_dentist + id_service | id_dentist, id_service | Tabla puente: servicios que presta cada odontólogo. |
| dentist_schedules | id_schedule | id_dentist | Horario semanal habitual. |
| dentist_blocks | id_block | id_dentist | Bloqueos puntuales (vacaciones, reuniones). |
| appointments | id_appointment | id_patient, id_dentist, id_service | Cita con inicio y fin, estado y precio de referencia. |
| appointment_logs | id_log | id_appointment, changed_by | Bitácora de cambios de estado de la cita. |
| medical_records | id_record | id_appointment (único) | Expediente de una atención. |
| notifications | id_notification | id_appointment | Confirmaciones y recordatorios de citas. |

## Relaciones y cardinalidad

*Tabla 6. Relaciones y cardinalidad.*

| Relación | Cardinalidad | Interpretación |
|---|---|---|
| roles — users | 1:N | Un rol agrupa varios usuarios; cada usuario tiene un rol. |
| users — patients | 1:0..1 | Un paciente puede tener una cuenta de acceso (opcional). |
| users — dentists | 1:0..1 | Cada odontólogo tiene una cuenta de acceso. |
| specialties — dentists | 1:N | Una especialidad agrupa varios odontólogos. |
| dentists — services | N:M | Resuelta con dentist_services: qué servicios brinda cada odontólogo. |
| dentists — dentist_schedules | 1:N | Un odontólogo tiene varios tramos de horario semanal. |
| dentists — dentist_blocks | 1:N | Un odontólogo puede tener varios bloqueos. |
| patients — appointments | 1:N | Un paciente tiene varias citas; cada cita pertenece a un paciente. |
| dentists — appointments | 1:N | Un odontólogo atiende muchas citas. |
| services — appointments | 1:N | El servicio define duración y precio; la cita conserva el precio de referencia. |
| appointments — appointment_logs | 1:N | Cada cambio de estado genera un registro. |
| users — appointment_logs | 1:N | Registra qué usuario hizo cada cambio. |
| appointments — medical_records | 1:0..1 | Una cita atendida produce un expediente. |
| appointments — notifications | 1:N | Una cita genera confirmación y recordatorios. |

## Diagramas de flujo de procesos principales

Se modelan dos procesos críticos del sistema. Ambos muestran las decisiones, las validaciones y la respuesta del sistema ante errores. Los pasos en naranja corresponden al usuario, los azules al sistema y los rojos a rutas de error.

### Agendamiento de una cita

El paciente o la recepción selecciona el servicio, el odontólogo y la fecha. El sistema combina el horario regular con los bloqueos y las citas existentes; si no hay espacio, ofrece alternativas. Con datos válidos, revalida el espacio dentro de una transacción para evitar reservas simultáneas, crea la cita, registra la bitácora y programa la notificación.

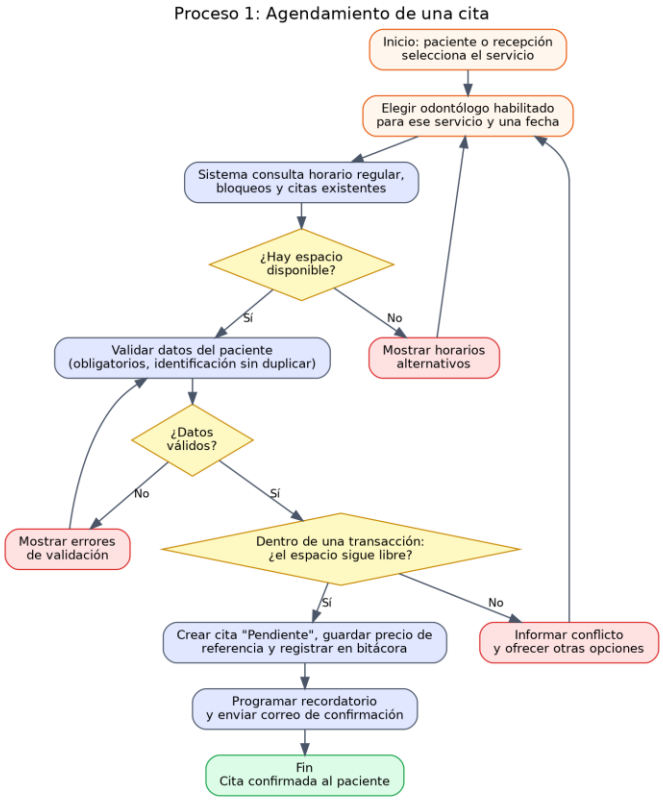



*Figura 3. Flujo de agendamiento de una cita.*

**Atención y expediente clínico**

El odontólogo abre la cita desde su agenda. Si el paciente no se presenta, la cita se marca como no asistida y queda en la bitácora. Si se presenta, revisa los antecedentes, documenta el examen, el diagnóstico y el plan de tratamiento, y el sistema valida los campos obligatorios antes de guardar el expediente y completar la cita.

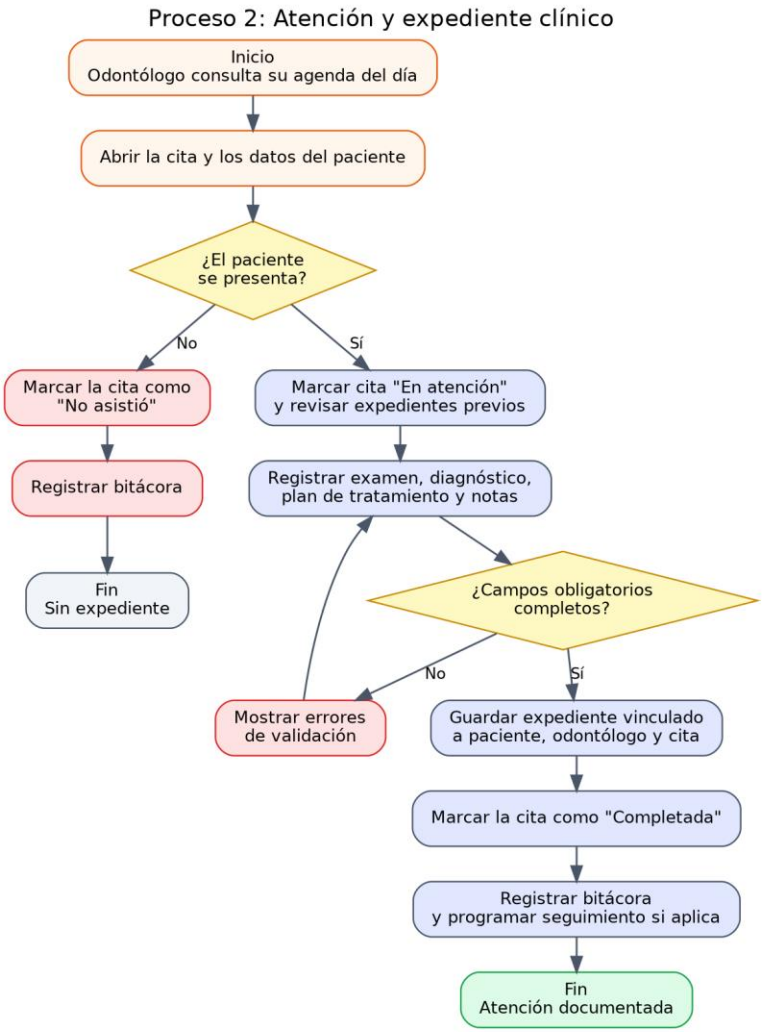



*Figura 4. Flujo de atención y expediente clínico.*

## Riesgos y medidas de control

*Tabla 7. Riesgos y medidas de control.*

| **Riesgo** | **Nivel** | **Respuesta** |
|---|---|---|
| Exposición de datos clínicos | Alto | Mínimo privilegio, secretos fuera del código, validación de entradas y logs sin datos sensibles. |
| Conflictos de horario | Alto | Validación dentro de una transacción y restricción de exclusión en la base de datos. |
| Alcance excesivo | Medio | Priorizar el flujo completo de cita y expediente; diferir lo que está fuera del alcance. |
| Ausencia de un integrante | Medio | Tareas visibles en el tablero, documentación mínima y revisión cruzada del código. |
| Diferencias entre ambientes | Medio | Docker, variables de entorno y pipeline de validación. |
| Límites de los planes gratuitos | Medio | Usar datos ficticios, monitorear el consumo y aprovechar el escalado a cero. |

## Anexo A. Script SQL del modelo de datos

Script de creación para PostgreSQL 15 o superior, probado en PostgreSQL 16. Se mantiene en el repositorio como database/schema.sql, junto con datos ficticios de prueba en database/seed.sql.

```
-- Sistema web para consultorios dentales - Modelo de datos (PostgreSQL 15+)
-- Ejecutar con un usuario que pueda crear extensiones (Supabase lo permite).


CREATE EXTENSION IF NOT EXISTS btree_gist;


-- ---------- Seguridad ----------
CREATE TABLE roles (
  id_role      smallint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name         varchar(30)  NOT NULL UNIQUE,
  description  varchar(200)
);


CREATE TABLE users (
  id_user        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_role        smallint     NOT NULL REFERENCES roles (id_role) ON DELETE RESTRICT,
  full_name      varchar(150) NOT NULL,
  email          varchar(255) NOT NULL UNIQUE,
  phone          varchar(30),
  password_hash  varchar(255) NOT NULL,
  status         varchar(10)  NOT NULL DEFAULT 'ACTIVE'
                 CHECK (status IN ('ACTIVE', 'INACTIVE')),
  created_at     timestamptz  NOT NULL DEFAULT now(),
  updated_at     timestamptz  NOT NULL DEFAULT now()
);


-- ---------- Pacientes ----------
CREATE TABLE patients (
  id_patient         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_user            bigint UNIQUE REFERENCES users (id_user) ON DELETE RESTRICT,
  full_name          varchar(150) NOT NULL,
  identification     varchar(30)  NOT NULL UNIQUE,
  birth_date         date,
  phone              varchar(30),
  email              varchar(255),
  emergency_contact  varchar(200),
  data_consent_at    timestamptz,
  status             varchar(10)  NOT NULL DEFAULT 'ACTIVE'
                     CHECK (status IN ('ACTIVE', 'INACTIVE')),
  created_at         timestamptz  NOT NULL DEFAULT now(),
  updated_at         timestamptz  NOT NULL DEFAULT now()
);


-- ---------- Odontologos y servicios ----------
CREATE TABLE specialties (
  id_specialty  smallint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name          varchar(100) NOT NULL UNIQUE,
  description   varchar(300)
);


CREATE TABLE dentists (
  id_dentist      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_user         bigint      NOT NULL UNIQUE REFERENCES users (id_user) ON DELETE RESTRICT,
  id_specialty    smallint    NOT NULL REFERENCES specialties (id_specialty) ON DELETE RESTRICT,
  license_number  varchar(30) NOT NULL UNIQUE,
  status          varchar(10) NOT NULL DEFAULT 'ACTIVE'
                  CHECK (status IN ('ACTIVE', 'INACTIVE')),
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now()
);


CREATE TABLE services (
  id_service        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
  name              varchar(120)  NOT NULL UNIQUE,
  description       varchar(500),
  duration_minutes  smallint      NOT NULL CHECK (duration_minutes > 0),
  price             numeric(10,2) NOT NULL CHECK (price >= 0),
  status            varchar(10)   NOT NULL DEFAULT 'ACTIVE'
                    CHECK (status IN ('ACTIVE', 'INACTIVE')),
  created_at        timestamptz   NOT NULL DEFAULT now(),
  updated_at        timestamptz   NOT NULL DEFAULT now()
);


CREATE TABLE dentist_services (
  id_dentist  bigint NOT NULL REFERENCES dentists (id_dentist) ON DELETE RESTRICT,
  id_service  bigint NOT NULL REFERENCES services (id_service) ON DELETE RESTRICT,
  PRIMARY KEY (id_dentist, id_service)
);


-- Horario semanal habitual (day_of_week: 1 = lunes ... 7 = domingo)
CREATE TABLE dentist_schedules (
  id_schedule  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_dentist   bigint   NOT NULL REFERENCES dentists (id_dentist) ON DELETE CASCADE,
  day_of_week  smallint NOT NULL CHECK (day_of_week BETWEEN 1 AND 7),
  start_time   time     NOT NULL,
  end_time     time     NOT NULL,
  CHECK (end_time > start_time),
  UNIQUE (id_dentist, day_of_week, start_time)
);


-- Bloqueos puntuales (vacaciones, reuniones, permisos)
CREATE TABLE dentist_blocks (
  id_block    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_dentist  bigint      NOT NULL REFERENCES dentists (id_dentist) ON DELETE CASCADE,
  starts_at   timestamptz NOT NULL,
  ends_at     timestamptz NOT NULL,
  reason      varchar(255),
  CHECK (ends_at > starts_at)
);


-- ---------- Citas ----------
CREATE TABLE appointments (
  id_appointment  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_patient      bigint        NOT NULL REFERENCES patients (id_patient) ON DELETE RESTRICT,
  id_dentist      bigint        NOT NULL,
  id_service      bigint        NOT NULL,
  starts_at       timestamptz   NOT NULL,
  ends_at         timestamptz   NOT NULL,
  status          varchar(15)   NOT NULL DEFAULT 'PENDING'
                  CHECK (status IN ('PENDING', 'CONFIRMED', 'COMPLETED', 'CANCELLED', 'NO_SHOW')),
  price_snapshot  numeric(10,2) NOT NULL CHECK (price_snapshot >= 0),
  created_at      timestamptz   NOT NULL DEFAULT now(),
  updated_at      timestamptz   NOT NULL DEFAULT now(),
  CHECK (ends_at > starts_at),
  -- La cita solo puede usar un servicio que ese odontologo tenga habilitado
  FOREIGN KEY (id_dentist, id_service)
    REFERENCES dentist_services (id_dentist, id_service) ON DELETE RESTRICT,
  -- Un odontologo no puede tener dos citas activas que se traslapen
  CONSTRAINT appointments_no_overlap
    EXCLUDE USING gist (id_dentist WITH =, tstzrange(starts_at, ends_at) WITH &&)
    WHERE (status <> 'CANCELLED')
);


CREATE TABLE appointment_logs (
  id_log           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_appointment   bigint      NOT NULL REFERENCES appointments (id_appointment) ON DELETE RESTRICT,
  changed_by       bigint      NOT NULL REFERENCES users (id_user) ON DELETE RESTRICT,
  previous_status  varchar(15),
  new_status       varchar(15) NOT NULL,
  changed_at       timestamptz NOT NULL DEFAULT now(),
  remarks          varchar(500)
);


CREATE TABLE medical_records (
  id_record       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_appointment  bigint      NOT NULL UNIQUE REFERENCES appointments (id_appointment) ON DELETE RESTRICT,
  symptoms        text,
  physical_exam   text,
  diagnosis       text,
  treatment_plan  text,
```

```
  prescriptions   text,
  clinical_notes  text,
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now()
);


CREATE TABLE notifications (
  id_notification  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_appointment   bigint       NOT NULL REFERENCES appointments (id_appointment) ON DELETE RESTRICT,
  type             varchar(20)  NOT NULL CHECK (type IN ('CONFIRMATION', 'REMINDER', 'CANCELLATION')),
  channel          varchar(10)  NOT NULL DEFAULT 'EMAIL' CHECK (channel IN ('EMAIL')),
  recipient        varchar(255) NOT NULL,
  status           varchar(10)  NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'SENT', 'FAILED')),
  retry_count      smallint     NOT NULL DEFAULT 0,
  scheduled_for    timestamptz  NOT NULL,
  sent_at          timestamptz,
  created_at       timestamptz  NOT NULL DEFAULT now()
);


-- ---------- Indices de apoyo ----------
CREATE INDEX idx_appointments_dentist_start ON appointments (id_dentist, starts_at);
CREATE INDEX idx_appointments_patient_start ON appointments (id_patient, starts_at);
CREATE INDEX idx_appointment_logs_appointment ON appointment_logs (id_appointment);
CREATE INDEX idx_notifications_pending ON notifications (scheduled_for) WHERE status = 'PENDING';
```